

Chapter 1

Loops

The programs we have seen so far are *straight-line code*; that is, they execute one instruction after another from top to bottom. This chapter introduces one of the most important programming-language features, the `for` loop, which allows simple programs to perform complex, repetitive tasks. This chapter also introduces the mathematical concepts of sequence and series, and a process for writing programs, incremental development.

We'll start by reviewing the exercise from the previous chapter; if you didn't do it, you might want to take a look before you go on.

1.1 Updating Variables

In Exercise 2.2, I asked you to write a program that models a bike-share system with bikes moving between two stations. Each day 5 percent of the bikes in Monterey are dropped off in Pacific Grove, and 3 percent of the bikes in Pacific Grove get dropped off in Monterey.

To update the state of the system, you might have been tempted to write something like

```
m = m - 0.05*m + 0.03*pg
pg = pg + 0.05*m - 0.03*pg
```

But that would be wrong, so very wrong. Why? The problem is that the first line changes the value of `m`, so when the second line runs, it gets the old value of `pg` and the new value of `m`. As a result, the change in `m` is not always the same as the change in `pg`, which violates the Principle of Conservation of Bikes!

One solution is to use temporary variables like `m_new` and `pg_new`:

```
m_new = m - 0.05*m + 0.03*pg
pg_new = pg + 0.05*m - 0.03*pg
m = m_new
pg = pg_new
```

This has the effect of updating the variables *simultaneously*; that is, it reads both old values before writing either new value.

The following is an alternative solution that has the added advantage of simplifying the computation:

```
m_to_pg = 0.05*m - 0.03*pg
m = m - m_to_pg
pg = pg + m_to_pg
```

It's easy to look at this code and confirm that it obeys Conservation of Bikes. Even if the value of `m_to_pg` is wrong, at least the total number of bikes is right. And that brings us to the Fifth Theorem of Debugging:

The best way to avoid a bug is to make it impossible.

In this case, removing redundancy also eliminates the opportunity for a bug.

1.2 Bug Taxonomy

The more you understand bugs, the better you will be at debugging. Here are a few common types of bus:

Syntax error You have written a command that cannot execute because it violates one of the language's syntax rules. For example, in MATLAB, you can't have two operands in a row without an operator, so `pi r^2` contains a syntax error, or you could forget the closing parenthesis. When the interpreter finds a syntax error, it prints an error message and stops running your program.

Runtime error Your program starts running, but something goes wrong along the way. For example, if you try to access a variable that doesn't exist, that's a runtime error. Division by zero is another example. When the interpreter detects the problem, it prints an error message and stops.

Logical error Your program runs without generating any error messages, but it doesn't do what you intended. The problem in the previous section, where we changed the value of `b` before reading the old value, is a logical error. Logical errors include some specific subspecies:

Semantic error The meaning inherent in the program is different from your intended meaning. For example, you use the command `sin(90)`, expecting MATLAB to interpret that as the sine of a 90° angle, but MATLAB returns 0.894 because the function expects the argument in radians.

Type error Errors involving incompatible data types. We'll see examples of this later, such as trying to add a string (letters) to numbers.

Numerical error Most computations in MATLAB are only approximately right. Most of the time the errors are small enough that we don't care, but in some cases the round-off errors are a problem.

Syntax errors are usually the easiest to deal with because you will get timely feedback on the error; the code simply won't run until they are fixed. Sometimes the error messages are confusing, but MATLAB can usually tell you where the error is, at least roughly. The MATLAB Editor can even continuously check your code before you try to execute, similar to automated spell-check when writing a document.

Runtime errors are a little harder because, as I mentioned before, MATLAB can tell you where it detected the problem, but often not what caused it.

Logical errors, akin to the garbage in, garbage out concept, are hard because MATLAB can't help at all. From MATLAB's point of view there's nothing wrong with the program; only you know what the program is supposed to do, so only you can check it. These errors are usually detected only through deliberate testing to execute the program, or parts of the program, when you have a clear way to verify that the actual behavior matches the intended outcome.

Semantic errors are best avoided at design time by reading the documentation about the commands you are using so that you are interpreting them the same way that MATLAB expects and writing down your models before implementing them in a program.

Numerical errors can be tricky because it's not clear whether the problem is your fault. For most simple computations, MATLAB produces the floating-point value that is closest to the exact solution, which means that the first 15 significant digits should be correct.

But some computations are ill-conditioned, which means that even if your program is correct, the round-off errors accumulate and the number of correct digits can be smaller. Sometimes MATLAB can warn you that this is happening, but not always! *Precision* (the number of digits in the answer) does not imply *accuracy* (the number of digits that are right).

1.3 Absolute and Relative Error

There are two ways of thinking about numerical errors. The first is *absolute error*, or the difference between the correct value and the approximation. We often write the magnitude of the error, ignoring its sign, when it doesn't matter whether the approximation is too high or too low.

The second way to think about numerical errors is *relative error*, where the error is expressed as a fraction (or percentage) of the exact value.

For example, we might want to estimate $9!$ using the formula $\sqrt{18\pi}(9/e)^9$. The exact answer is $9 \cdot 8 \cdot 7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 362,880$. The approximation is 359,536.87. So the absolute error is 3,343.13.

At first glance, that sounds like a lot—we're off by three thousand—but we should consider the size of the thing we are estimating. For example, \$3,000 matters a lot if we're talking about an annual salary, but not at all if we're talking about the national debt.

A natural way to handle this problem is to use relative error. In this case, we would divide

the error by 362,880, yielding 0.00921, which is just less than 1 percent. For many purposes, being off by 1 percent is good enough.

1.4 for Loops

Let's return to the bike-share example. In the previous chapter, we wrote a *bike_update.m* script to simulate a day in the life of a bike-share system. To simulate an entire month, we'd have to run the script 30 times. We could enter the same command 30 times, but it's simpler to use a *loop*, which is a set of statements that executes repeatedly.

To create a loop, we can use the `for` statement, like this:

```
for i=1:30
    bike_update
end
```

The first line includes what looks like an assignment statement, and it *is* like an assignment statement, except that it runs more than once. The first time, it creates the variable `i` and assigns it the value 1. The second time, `i` gets the value 2, and so on, up to and including 30.

The colon operator (`:`) specifies a *range* of integers. You can create a range at the prompt:

```
>> 1:5
ans = 1      2      3      4      5
```

The variable you use in the `for` statement is called the *loop variable*. It's common to use the names `i`, `j`, and `k` as loop variables.

The statements inside the loop are called the *body*. By convention, they are indented to show that they're inside the loop, but the indentation doesn't affect the execution of the program. The `end` statement marks the end of the loop.

To see a loop in action you can run one that displays the loop variable:

```
>> for i=1:5
    i
end

i = 1
i = 2
i = 3
i = 4
i = 5
```

As this example shows, you *can* run a `for` loop from the command line, but it's more common to put it in a script.

Exercise 1.1. Create a live script named *bike_loop.mlx* that does the following:

1. Assign the initial values (precondition) of `m = 100` and `pg = 100`.
2. Run the `for` loop from 1 to 30.
 - (a) Inside the loop, call the program `bike_update.m` to update the values of `m` and `pg`. The program should echo the values of `m` and `pg` after each update.

If everything goes smoothly, your script will display a long stream of numbers on the screen. It's probably too long to fit, and even if it did fit, it would be hard to interpret. A graph (coming soon) would be much better!

1.5 Plotting

If the output of your program is a long stream of numbers, it can be hard to see what is happening. Plotting the results can make things clearer.

The `plot` function is a versatile tool for plotting two-dimensional graphs. Unfortunately, it's so versatile that it can be hard to use (and hard to read the documentation). We'll start simple and work our way up.

To plot a single point, type

```
>> plot(1, 2, 'o')
```

A Figure Window should appear with a graph and a single blue circle at x position 1 and y position 2.

The letter in single quotes is a *style string* that specifies how the point should be plotted; `o` indicates a circle. Other shapes include `+`, `*`, `x`, `s` (for a square), `d` (for a diamond), and `^` (for a triangle).

You can also specify the color by starting the style string with a color code:

```
>> plot(1, 2, 'ro')
```

Here, `r` stands for red; the other colors include `g` for green, `b` for blue, `c` for cyan, `m` for magenta, `y` for yellow, and `k` for black.

When you use `plot` this way, it can only plot one point at a time. If you run `plot` again, it clears the figure before making the new plot. The `hold` command lets you override that behavior: `hold on` tells MATLAB not to clear the figure when it makes a new plot; `hold off` returns to the default behavior.

Try this:

```
>> clf
>> hold on
>> plot(1, 1, 'ro')
>> plot(2, 2, 'go')
>> plot(3, 3, 'bo')
>> xlabel('x axis label [units]')
```

```
>> ylabel('y axis label [units]')  
>> hold off
```

The `clf` command clears the figure before we start plotting.

If you run the code above, you should see a figure with three circles and text labels on the axes similar to Figure 1.1. MATLAB scales the plot automatically so that the axes run from the lowest values in the plot to the highest.

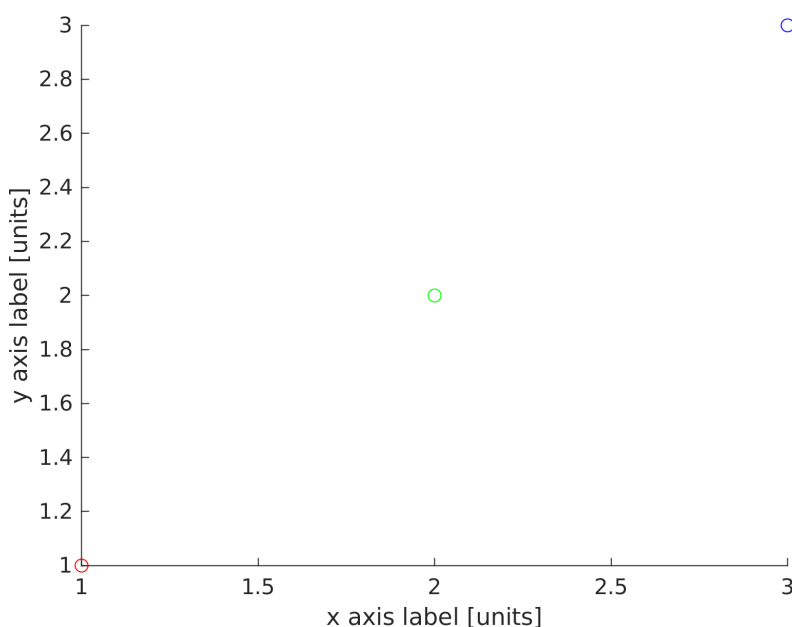


Figure 1.1: Simple figure with three markers and axes labels.

Exercise 1.2. Using *bike_loop.mlx* as a starting point, develop a live script *bike_loop_plot.mlx* so that it clears the figure before running the loop. Then, each time through the loop, it should plot the value of `m` versus the value of `ii` with a red circle.

Once you get that working, modify it so it plots the values of `pg` with blue diamonds.

1.6 Sequences

Now that we have the ability to write loops, we can use them to explore sequences and series, which are useful for describing and analyzing systems that change over time.

In mathematics, a *sequence* is a set of numbers that corresponds to the positive integers. The numbers in the sequence are called *elements*. In math notation, the elements are denoted with subscripts, so the first element of the series A is A_1 , followed by A_2 , and so on.

A `for` loop is a natural way to compute the elements of a sequence. As an example, in a geometric sequence, each element is a constant multiple of the previous element. Expressed mathematically, element $i + 1$ (the next element) of the sequence can be expressed as the

product of the element i (the previous element) and the constant *common ratio*, r or

$$A_{i+1} = A_i r.$$

As a more specific example, let's look at the sequence with $A_1 = 1$ and $r = 1/2$ so that the relationship $A_{i+1} = A_i/2$, for all i . In other words, each element is half as big as the one before it.

The following loop computes the first 10 elements of A :

```
a = 1
r = 1/2
for ii = 2:10
    a = a * r
end
```

The first line initializes the variable `a` with the first element of the sequence, A_1 . Each time through the loop, we find the next value of `a` by dividing the previous value by 2, and assign the result back to `a`. At the end, `a` contains the 10th element. The reason we use the variable `ii` instead of just `i` is that MATLAB defines both `i` and `j` as complex numbers, so using either as a variable causes a naming collision that may be tough to debug.

The other elements are displayed on the screen, but they are not saved in a variable. Later, we'll see how to store the elements of a sequence in a vector.

This loop computes the sequence *recurrently*, which means that each element depends on the previous one. For this sequence, it's also possible to compute the i th element *directly*, as a function of ii , without using the previous element. In math notation, $A_i = A_1 r^{ii-1}$.

Exercise 1.3. Write a script named *sequence.m* that uses a loop to compute elements of A directly.

1.7 Series

In mathematics, a *series* is the sum of the elements of a sequence. It's a terrible name, because in common English, "sequence" and "series" mean pretty much the same thing, but in math, a sequence is a set of numbers, and a series is an expression (a sum) that has a single value. In math notation, a series is often written using the summation symbol $\sum$.

For example, the sum of the first 10 elements of A is

$$\sum_{i=1}^{10} A_i$$

A `for` loop is a natural way to compute the value of this series:

Listing 1.1: A program that calculates a simple series

```
A1 = 1;
total = 0;
for ii = 1:10
    a = A1 * (1/2)^(ii-1);
    total = total + a;
end
ans = total
```

Let's walk through what's happening here. `A1` is the first element of the sequence, so we assign it to be 1; we also create `total`, which will store the cumulative sum. Each time through the loop, we set `a` to the i th element and add `a` to the `total`. At the end, outside the loop, we store `total` as `ans`.

The way we're using `total` is called an *accumulator*, that is, a variable that accumulates the result a little bit at a time.

Exercise 1.4. This example computes the terms of the series directly. As an exercise, write a script named *series.m* that computes the same sum by computing the elements recurrently. You will have to be careful about where you start and stop the loop.

1.8 Generalization

As written, the previous example always adds up the first 10 elements of the sequence, but we might be curious to know what happens to `total` as we increase the number of terms in the series. If you've studied geometric series, you might know that this series converges on 2; that is, as the number of terms goes to infinity, the sum approaches 2 asymptotically.

To see if that's true for our program, we can replace the constant 10 in Listing 1.1 with a variable named `n`:

Listing 1.2: Updating our code from Listing 1.1 to have a variable number of terms

```
A1 = 1;
total = 0;
for i=1:n
    a = A1 * (1/2)^(i-1);
    total = total + a;
end
ans = total
```

The code in Listing 1.2 can now compute any number of terms, with the precondition that you have to set `n` before you execute the code. I put this code in a file named *series.m*, then ran it with different values of `n`:

```
>> n=10; series
total = 1.998046875000000

>> n=20; series
```

```
total = 1.99999809265137

>> n=30; series
total = 1.99999999813735

>> n=40; series
total = 1.99999999999818
```

It sure looks like it's converging on 2.

Replacing a constant with a variable is called *generalization*. Instead of computing a fixed, specific number of terms, the new script is more general; it can compute any number of terms. This is an important idea we'll come back to when we talk about functions.

1.9 Incremental Development

As you start writing longer programs, you might find yourself spending more time debugging. The more code you write before you start debugging, the harder it is to find the problem.

Incremental development is a way of programming that tries to minimize the pain of debugging by developing and testing in small steps. The fundamental steps are:

1. Always start with a working program. If you have an example from a book, or a program you wrote that is similar to what you are working on, start with that. Otherwise, start with something you *know* is correct, like `x = 5`. Run the program and confirm that you are running the program you think you are running. This step is important, because in most environments there are little things that can trip you up when you start a new project. Get them out of the way, so you can focus on programming.
2. Make one small, testable change at a time. A *testable* change is one that displays something on the screen (or has some other effect) that you can check. Ideally, you should know what the correct answer is or be able to check it by performing another computation.
3. Run the program and see if the change worked. If so, go back to step 2. If not, you'll have to do some debugging, but if the change you made was small, it shouldn't take long to find the problem.

With incremental development, your code is more likely to work the first time, and if it doesn't, the problem is more likely to be obvious. And that brings us to the Sixth Theorem of Debugging:

The best kind of debugging is the kind you don't have to do.

In practice, there are two problems with incremental development. First, sometimes you have to write extra code to generate visible output that you can check. This extra code is called *scaffolding* because you use it to build the program and then remove it when you are done. But the time you save on debugging is almost always worth the time you invest in scaffolding.

The second problem is that when you're getting started, you might not know how to choose steps that get from `x = 5` to the program you're trying to write. We'll look at an extended example in Chapter 6.

If you find yourself writing more than a few lines of code before you start testing, and you're spending a lot of time debugging, you should try incremental development.

1.10 Chapter Review

In this chapter, we used a loop to perform a repetitive computation—updating a model 30 times—and to compute sequences and series. Also, we used the `plot` function to visualize the results.

Here are some terms from this chapter you might want to remember.

Absolute error is the difference between an approximation and an exact answer. *Relative error* is the same difference expressed as a fraction or percentage of the exact answer.

A *loop* is part of a program that runs repeatedly. A *loop variable* is a variable that gets assigned a different value each time through the loop. A *range* is a sequence of values assigned to the loop variable, often specified with the colon operator—for example, `1:5`. The *body* of a loop is the set of statements inside the loop that runs repeatedly. An *accumulator* is a variable that is used to accumulate a result a little bit at a time.

In mathematics, a *sequence* is a set of numbers that correspond to the positive integers. The numbers that make up the sequence are called *elements*. A *series* is the sum of a sequence of elements. Sometimes we compute the elements of a sequence *recurrently*, which means that each new element depends on previous elements. Sometimes we can compute the elements *directly*, without using previous elements.

Generalization is a way to make a program more versatile, for example, by replacing a specific value with a variable that can have any value. *Incremental development* is a way of programming by making a series of small, testable changes. *Scaffolding* is code you write to help you program or debug, but that is not part of the finished program.

In the next chapter, we'll use a vector to store the results from a loop.

1.11 Exercises

Before you go on, you might want to work on the following exercises.

Exercise 1.5. Years ago I was in a fudge shop and saw a sign that said “Buy one pound of fudge, get another quarter pound free.” That's simple enough.

But if I ran the fudge shop, I would offer a special deal to anyone who could solve the following problem:

If you buy a pound of fudge, we'll give you another quarter pound free. And then we'll give you a quarter of a quarter pound, or one-sixteenth. And then we'll give you a quarter of that, and so on. How much fudge would you get in total?

Write a script called *fudge.m* that solves this problem. Hint: start with *series.m* and generalize it by replacing the ratio $1/2$ with a variable, `r`.

Exercise 1.6. We have already seen the Fibonacci sequence, F , which is defined recurrently as

$$\text{for } i \geq 3, \quad F_i = F_{i-1} + F_{i-2}$$

In order to get started, you have to specify the first two elements, but once you have those, you can compute the rest. The most common Fibonacci sequence starts with $F_1 = 1$ and $F_2 = 1$.

Write a script called *fibonacci2.m* that uses a `for` loop to compute the first 10 elements of this Fibonacci sequence. As a postcondition, your script should assign the 10th element to `ans`.

Now generalize your script so that it computes the n th element for any value of `n`, with the precondition that you have to set `n` before you run the script. To keep things simple for now, you can assume that `n` is greater than 0.

Hint: you'll have to use two variables to keep track of the previous two elements of the sequence. You might want to call them `prev1` and `prev2`. Initially, `prev1` = F_1 and `prev2` = F_2 . At the end of the loop, you'll have to update `prev1` and `prev2`; think carefully about the order of the updates!